Conversation folding in llm-gateway

A report of the work done in one session, 22–24 October 2026: from folding receipts to a gateway-owned fact ledger, then a second quality axis, task success. Every number here comes from the benchmark result files committed in the repository.

In short. Folding replaces old conversation turns with a short summary so each request sends fewer tokens. Today it gained receipts, a benchmark against real models, and a design that keeps facts correct when they change. **On stable facts:** every fact kept, at a 47–54% saving even after prompt caching. **On changing facts:** with a gateway-owned fact ledger, no folded answer to a changed fact was wrong in the final rounds, because the gateway refuses any summary it cannot verify. With gpt-4.1-mini summarizing and the cost check set to how often a summary is really reused, two runs full of corrections kept all 889 facts at a 54–55% saving, through the fold cap, a correction and a re-fold. With the cheaper gpt-4.1-nano almost every fold was refused, so those conversations saved nothing. **On task success:** a new benchmark checks standing rules given in the conversation; folding first lost an answer-format rule (folded success 13% against 32% unfolded), and with rules kept word for word in the ledger the folded model followed them better than the unfolded one (58–60% against 38–42%). 44 benchmark runs, 5,179 paired calls, $8.67 at provider prices.

1. Folding receipts

The day began by writing two plans for folding and finishing the receipts plan after its four decisions (D1–D4) were approved. It was built in three phases, one commit each, and merged as **PR 53**:

- the summary the model was sent comes back to the caller in `x-gateway-fold-summary`, and the main receipt binds the folded request (`folded_request_digest`);
- the summarizer call gets its own allow and outcome receipt, so a fold is evidence in its own right;
- background folding is refused for receipts tenants, which have no caller to hand the salt to;
- a request refused after its fold still returns the salt and fold headers, so the caller can check what it paid for.

A third reviewer's findings followed in **PR 55**: a tenant's own `allowed_models` is enforced at runtime (and for its summarizer), refusals after a fold return the salt, and the benchmark's HTTP client no longer follows redirects, so an API key cannot be sent elsewhere.

2. A benchmark against real models

`tools/fold_retention.py` sends every call twice, to a tenant without folding and one with it, same model, same messages, temperature 0, and asks for ten facts that the conversation states, corrects and confuses with a second project. Answers are scored by strict typed equality; a fact counts only if it was folded away and the unfolded tenant got it right. Groq's daily token limit stopped the first runs, so the benchmark moved to OpenRouter (gpt-4.1-mini answering, gpt-4.1-nano summarizing): **PR 54**.

3. Does folding still pay with prompt caching?

The gateway bills every prompt token at full price, but OpenAI serves a repeated prefix from its cache at a quarter of it, and an unfolded conversation, which only grows at its end, gets most of that discount. The harness learned to record cached tokens, price them at the cached rate, alternate which tenant goes first, and run each profile twice (**PR 57**, with a dashboard and the first PDF report):

Run	Facts kept	Cached share, unfolded / folded	Cost with caching	Saving
fixed, 200 calls	1492 / 1492	59% / 19%	$0.259 → $0.120	54%
fixed, repeat	1492 / 1492	64% / 20%	$0.243 → $0.118	51%
mutating, 120 calls	772 / 816	57% / 15%	$0.136 → $0.066	52%
mutating, repeat	836 / 867	65% / 5%	$0.123 → $0.060	51%

Caching does not erase the saving. But both mutating runs lost the latest *owner* right after a fold and never got it back, and that started the rest of the day.

4. Chasing the lost owner

First, diagnostics (**PR 58**): the result files now keep what each tenant answered when wrong, every summary the folded tenant was sent, each call's fold outcomes, and which checks a rejected summary failed. The first summaries showed the cause at once: a chronological story that stated the old owner first, the correction later, and was cut at 1,200 characters

exactly where the newest corrections sat. Then one safeguard per round, each two mutating runs of 120 calls:

Round	Facts kept	Saving	What it showed
Current-facts block and field/value pins	667/667, 705/818	42%, 45%	The summarizer read “Noor replaces Mira” backwards
A replaced value may not be current	640/647, 695/695	36%, 47%	A stated-once port was dropped
Bare numbers pinned; rejection reasons	452/452, 727/727	22%, 37%	Old values became mandatory; folding stalled
Replaced values not required	488/488, 510/510	27%, 31%	The first fold took 8 attempts
One corrective retry	793/811, 662/749	48%, 46%	Folding flowed; two lines for one fact
Same, gpt-4.1-mini summarizing	715/715, 720/720	43%, 40%	Clean, but few folds at its price

Every safeguard closed the failure it targeted and exposed the next. Strict checks lost nothing but stopped folding; looser ones let the summarizer's mistakes through. Two reviews pasted during the day named the root: the gateway was inferring each fact's current value from the summary's wording.

5. The fact ledger

PR 59 moves the facts out of the summary. The summarizer now returns JSON: fact updates from the new turns, each a field, a value and the exact sentence it came from, plus a short context note. The gateway keeps the facts per conversation and writes the `Current facts` block itself. An update is applied only if:

- its evidence is one sentence from the new turns, holds the value, and names the fact;
- its value is not the one its own sentence replaces (“Noor replaces Mira” can never set Mira);
- it is not a second name for a kept fact, and goes to the kept fact its sentence names;
- afterwards, no kept fact still holds a value the new turns replace.

Updates apply in the conversation's order, not the reply's. Each fact keeps its source sentence, turn and last eight replaced values. A later fold only adds updates, so it cannot drop or flip a fact. One failed update fails the whole reply, which gets one corrective retry; the back-off after a rejection doubles each time, so a conversation that cannot be folded stops paying for attempts.

Ledger version	Run	Facts kept	Folds	Saving
First	fixed, 200 calls	1441 / 1441	5 of 16	54%
First	mutating ×2	625 / 706; none	4 of 10; 0 of 15	49%; –15%
Evidence names its fact	mutating ×2	417 / 417, 421 / 421	1 of 14, 1 of 13	22%, 21%
Same, gpt-4.1-mini	mutating	710 / 710	2 of 2	41%
Same	fixed, 200 calls	1223 / 1347	7 of 14	55%
+ stale facts refused	fixed, 200 calls	1417 / 1417	4 of 18	47%
Same	mutating ×2	none	0 of 15, 0 of 15	–14%, –16%
+ growing back-off	mutating ×2	none	0 of 4, 0 of 4	–3%, –3%

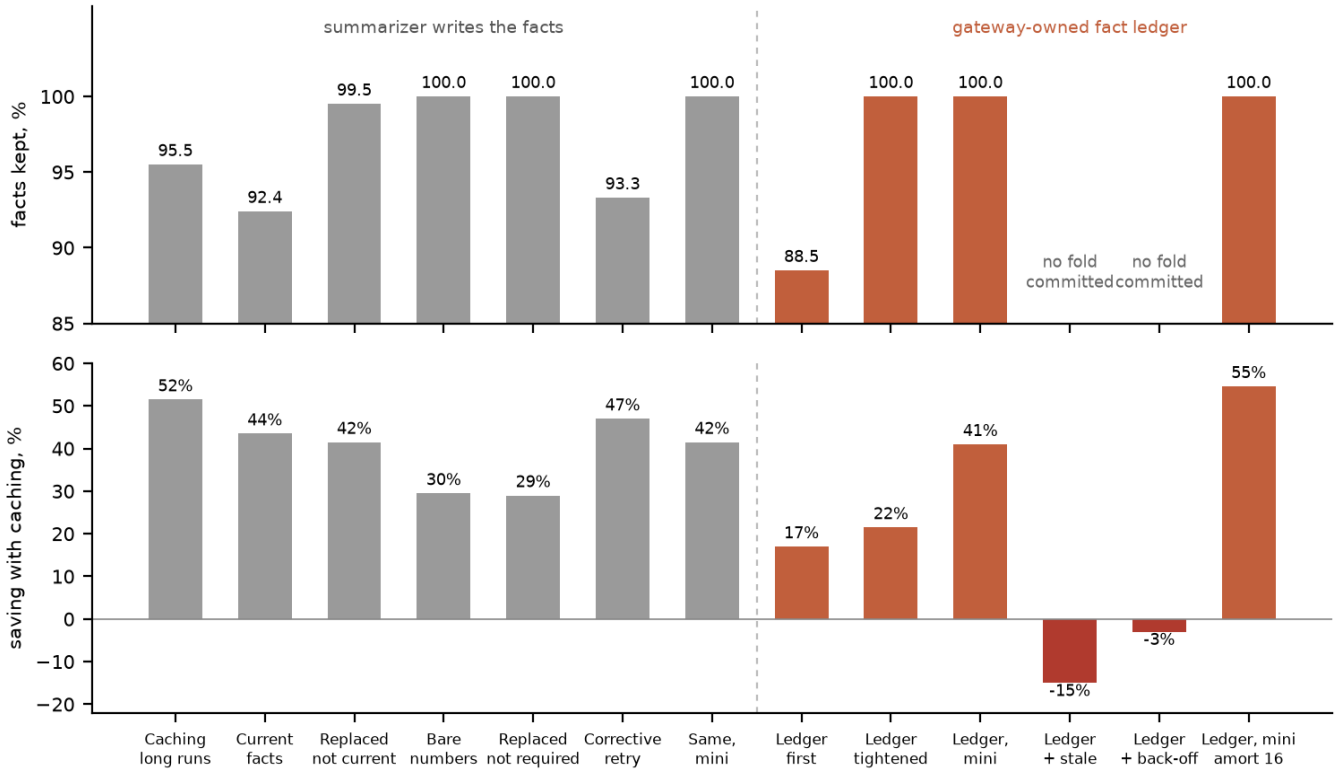
The last three runs used gpt-4.1-mini, whose updates were never refused, and varied the cost check's `amortization_requests`, the number of requests a fold's saving is counted over. In these runs a fold was really reused for about 21 calls:

amortization_requests	Folds	Facts kept	Saving	Reached
4 (the profiles' setting)	2	710 / 710	41%	warm
8	4	863 / 863	49%	warm
16	6	889 / 889	55%	re-folded
16, repeat	6	889 / 889	54%	re-folded

At 16 both runs reached the fold cap at call 85, were sent “owner is now Ines” at call 86, and re-folded at call 106 with *owner: Ines (was Noor)*; the owner was right on every counted call after it.

The first ledger let two kinds of wrong data in: a value taken from a turn that stated two facts, and one fact under two names. The tightened ledger then let a correction start a new fact beside the old one (*ceiling: 1800* beside *spending ceiling usd: 2400*). Each became a rule with its own test.

Every round on the mutating scenario



Facts kept counts only facts folded away. Grey: the summarizer writes the facts. Orange: the gateway keeps them. Red: the folded tenant cost more than the unfolded one.

6. A second quality axis: task success (24 October)

Fact recall is not the whole job: a summary can keep every fact and still drop “never mention BIRCH-204”. **PR 60** adds `tools/fold_tasks.py`: the same paired calls on the mutating conversation, with five standing rules given in its user turns, checked on every answer by a deterministic checker for both tenants:

- answer in at most three “- ” bullets;
- never mention BIRCH-204 or Ana;
- end every answer with *Ref: CEDAR-731*, changed at call 60 to *Ref: CEDAR-731/B*;
- from call 15, write dates as DD.MM.YYYY.

A call succeeds when it obeys every rule in force. The report puts both tenants' task success rate next to the saving; the run fails if the folded tenant breaks a rule that was folded away and that the unfolded tenant obeyed, or, with `--max-success-drop`, if its success rate falls further than that below the unfolded one.

The first runs showed a real cost of folding. The summarizer kept the bullets rule only as a sentence of its context note, and the folded model broke it; the closing line and the date format, which it happened to file as ledger facts, were obeyed on every call. **PR 61** therefore keeps standing instructions as their own kind of ledger entry: the exact sentence from the conversation, in the user's own wording, shown first and never cut; a rule change must name the rule it replaces; and every instruction-like sentence in the folded turns must end up kept or replaced, so no rule survives only as a paraphrase.

gpt-4.1-mini summarizing, amortization 16	Bullets rule broken	Task success, unfolded → folded	Saving
Before (PR 60), run 1	30 times	32% → 13%	58%
Before, run 2	10 times	40% → 53%	54%
After (PR 61), run 1	once	38% → 60%	43%
After, run 2	twice	42% → 58%	54%

With its rules word for word at the top of the summary, the folded model followed them better than the unfolded one, whose rules sat at turn 8 of a long conversation (bullets 98% against 85–90%). Two caveats: the unfolded model itself breaks “never mention BIRCH-204” whenever the question asks which projects came up, which keeps the baseline low (32–42%); and format compliance is noisy, so the two runs of each version differ widely while the saving repeats. gpt-4.1-nano still committed no fold.

7. How the work was tested

- The full Rust suite (595 tests across 11 test binaries) and clippy with warnings as errors ran after every change; the Python tools have 71 tests.
- Every new rule was broken on purpose, one at a time, to confirm a test fails: 121 deliberate breaks across the diagnostics, safeguards, ledger and task-success work. Three times a break went unnoticed; each gap was closed with a test.
- Before spending on a run, new checks were replayed against real summaries from earlier runs, and the benchmark's own sentences were run through the ledger, to catch false rejections.
- Result files were checked for summed spends, the scoring rule, both call orders, and the absence of key-like strings. CI passed on Linux and Windows for every merged PR and for PR 58; on PR 59 the receipts demo first failed, because its mock model still answered the summarizer in plain text, and was fixed and rerun locally.

8. What is established, and what is not

Established	Not yet established
Folding saves 47–54% on stable facts, prompt caching included, with every fact kept.	That a cheap summarizer can do it: with gpt-4.1-nano almost every fold of a changing conversation was refused.
With the ledger, the summarizer's mistakes become refused folds, not wrong answers, in every final run.	Other conversations: one synthetic scenario of ten facts and one distractor project.
With gpt-4.1-mini and amortization_requests 16: 54–55% saving with every changed fact kept, through a correction and a re-fold, twice.	Stable results over many repeats: most rounds were two runs.
A conversation that cannot be folded costs about 3% extra, not 15%; the response's x-request-id is the one in the logs.	The right amortization_requests in general: it depends on how long conversations run after a fold.
Standing rules kept word for word: folded task success 58–60% against 38–42% unfolded; the format rule broken once or twice instead of 30 and 10 times.	Rules worded in ways the instruction detector does not know (no “standing rule”, “from now on”, “always”, “never”), and tool calls, which folding still refuses to touch.

9. Closed, and still open

- **Closed: x-request-id.** The middleware no longer replaces the handler's id, so a response's id is the one in the decision log and the receipts; a client's own id is echoed and logged as client_request_id. The benchmark now attaches every call's fold decision by it.
- **Closed: a correction followed by a re-fold,** in two real runs (above).
- **Closed: the cost check with a costlier summarizer,** measured at 4, 8 and 16; the README says to set it near real reuse.
- **Closed: a quality axis.** Task success for standing rules (PR 60), and the ledger fix it pointed to (PR 61).
- **Open: merge PR 60, then PR 61** (stacked; PRs 58 and 59 are merged).
- **Open:** conversations beyond the benchmark's one scenario, and whether a summarizer cheaper than mini can report facts reliably enough.
- **Open:** the dashboard built for PR 57 must be shared from its Share menu before others can open it.

Appendix: pull requests

PR	What	State
53	Folding receipts: summary header, folded-request digest, fold allow and outcome	merged
54	Fold-retention benchmark on OpenRouter	merged
55	Reviewer fixes: tenant allowlist at runtime, salt after a refused fold, no redirects	merged
57	Prompt-caching costs, alternating arms, repeated long runs, dashboard and first report	merged
58	Diagnostics, safeguards for changing facts, and documented limits	merged
59	The fact ledger, a growing back-off, the x-request-id fix, and the amortization runs	merged
60	Task-success benchmark: standing rules checked on every answer	open
61	Standing instructions as their own kind of ledger entry, kept word for word	open

Benchmark totals for the session: 44 runs (41 OpenRouter, 3 Groq), 5,179 paired calls. Provider cost with caching for the runs that recorded it: $8.67. Gateway-billed spend for both tenants, at full prompt price, is higher: it charges every prompt token at the full rate.